

Sumarização Extrativa de Artigos da Wikipédia Utilizando Grafos e PageRank

Disciplina: EDA2 – Estrutura de Dados e Algoritmos

Membros do Grupo:

JOÃO EDUARDO PEREIRA RABELO/ 18-0053299

LEONARDO RAMIRO ALVES DE OLIVEIRA/ 22-1031336

THIAGO MELO TONIN/ 22-1022453

TIAGO ANTUNES BALIEIRO/ 23-1011838

Problema Encontrado

- Artigos da Wikipédia são muito usados por estudantes, porém são longos e densos.
- Ler o conteúdo completo demanda muito tempo.
- Estudantes não possuem muito tempo
- Objetivo: Gerar um resumo automático que capture as ideias principais, mantendo a coesão e evitando repetições.

Solução proposta: Sistema que auxilia na educação, transformando o artigo em um grafo de frases, aplica PageRank para identificar as mais relevantes e monta um resumo extrativo.

Abordagem da Ideia

- O artigo é dividido em frases.
- Cada frase se torna um vértice do grafo.
- Arestas conectam frases com similaridade semântica.
- Pesos medem o grau de similaridade.
- O PageRank é executado sobre o grafo para ranquear as frases.
- As frases com maior pontuação compõem o resumo, mantendo a ordem original.

Frase-chave: Uma frase é importante se está fortemente conectada a outras frases relevantes.

Entrada e Saída de Dados

- ***Entrada:***
 - URL da Wikipedia ou título do artigo.
 - Parâmetros: idioma (pt/en), método de similaridade, número de frases, etc.
- ***Saída:***
 - Resumo textual (extrativo).
 - Métricas do grafo (número de vértices, arestas, densidade).
 - Visualização opcional do grafo.
 - Lista de frases com suas pontuações.

Vértices/Frases

- Cada vértice representa uma frase completa do artigo.
- A segmentação é feita com spaCy (preprocessor .py), que identifica os limites de sentença.

Exemplo:

```
V1 = "A inteligência artificial é uma área da ciência da computação."
```

```
V2 = "Ela busca criar sistemas capazes de realizar tarefas inteligentes."
```

```
V3 = "Esses sistemas são usados em reconhecimento de voz, visão computacional e tradução automática."
```

Por que frases?

- A sumarização extrativa seleciona trechos já existentes; frases são unidades completas que preservam o sentido.

Arestas e Pesos

- Duas frases são conectadas se a similaridade entre elas ultrapassa um limiar.
- O peso da aresta é o valor da similaridade (entre 0 e 1).

Exemplo:

```
sim(V1, V2) = 0.72 → aresta V1-V2 com peso 0.72
```

```
sim(V1, V3) = 0.30 → abaixo do limiar → sem aresta
```

- O grafo é não direcionado porque a similaridade é simétrica. Para o PageRank, tratamos cada aresta como bidirecional.

Cálculo de Similaridade

Abordagem principal: Embeddings semânticos

- Comparam a frase pelo significado
- Cada frase é convertida em vetor e a similaridade é calculada pelo cosseno entre esses vetores, definindo o peso das arestas do grafo

Modelo usado:

```
sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
```

TF-IDF e Jaccard

- Como comparação, também usamos TF-IDF + cosseno e Jaccard. O TF-IDF serve como baseline lexical ponderado, enquanto o Jaccard mede a sobreposição direta de palavras.

Pré-processamento com spaCy - preprocessor.py

Antes de calcular a similaridade, o texto é processado com spaCy (***preprocessor.py***):

- Segmentação em frases.
- Tokenização.
- Remoção de pontuação e stopwords (ex.: “de”, “a”, “o”, “que”).
- Lematização: reduz palavras à sua forma base (ex.: “estudando” → “estudar”).

Exemplo:

```
def tokenize(self, text):  
    doc = self.nlp(text.lower())  
    return [token.lemma_ for token in doc  
            if not token.is_punct and not token.is_space and not token.is_stop]
```

Isso garante que a similaridade seja calculada sobre termos significativos.

Pré-processamento com spaCy

```
15 class TextPreprocessor:
16     """
17     Pré-processa texto usando spaCy com modelos treinados.
18     """
19
20     def __init__(self, lang: str = "pt"):
21         self.lang = lang
22         # Primeiro carrega o modelo
23         if lang == "pt":
24             self.nlp = spacy.load("pt_core_news_sm")
25         else:
26             self.nlp = spacy.load("en_core_web_sm")
27         # Depois configura as stopwords
28         self.stopwords = self.nlp.Defaults.stop_words
29
30     def tokenize(self, text: str) -> List[str]:
31         """Tokeniza e lematiza corretamente usando spaCy."""
32         doc = self.nlp(text.lower())
33         tokens = []
34         for token in doc:
35             # Filtra pontuação, espaços, stopwords
36             if not token.is_punct and not token.is_space and not token.is_stop:
37                 lemma = token.lemma_
38                 if len(lemma) > 2:
39                     tokens.append(lemma)
40         return tokens
```

Construção do Grafo

O grafo é construído pela classe ***SentenceGraph*** (*graph_summarizer.py*).

- Vértices: n frases.
- Arestas: adicionadas quando a similaridade $\geq$ limiar.

Limiar adaptativo (definido no código):

- Estratégia ***auto_density***: ajusta o limiar para que o grafo atinja uma densidade alvo (ex.: 12% dos pares possíveis).
- Estratégia ***mean_std***: usa média + desvio padrão das similaridades.
- Modo manual: valor fixo (ex.: 0.10).

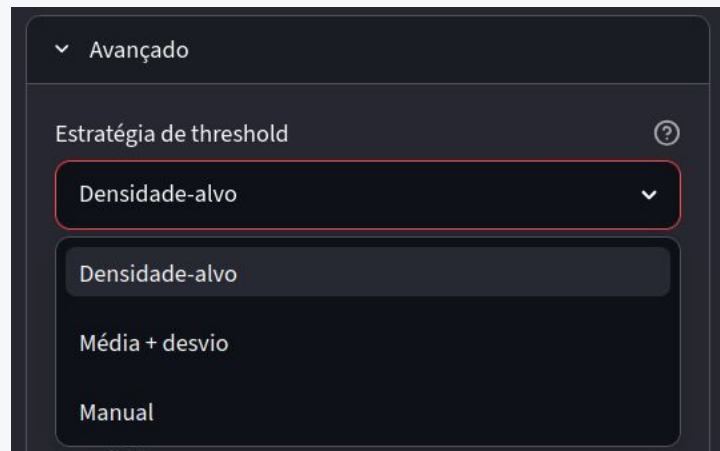
Também suportamos top-K: conectar cada vértice apenas às K frases mais similares (não usado no modo padrão, mas disponível).

Construção do Grafo - graph_summarizer.py

```
192 class SentenceGraph:
193     """
194     Grafo não-direcionado e ponderado onde:
195     - Vértices (nós): índices das frases
196     - Arestas: frases com similaridade > threshold
197     - Peso: valor da similaridade (Jaccard ou TF-IDF cosseno)
198
199     Representação interna: lista de adjacência (dict de dicts)
200     """
201
202     def __init__(self, n_sentences: int):
203         self.n = n_sentences
204         # Lista de adjacência: adj[i][j] = peso da aresta
205         self.adj: Dict[int, Dict[int, float]] = {i: {} for i in range(n_sentences)}
206         self.n_edges = 0
207
208     def add_edge(self, i: int, j: int, weight: float):
209         """Adiciona aresta bidirecional com peso."""
210         if i != j and weight > 0:
211             self.adj[i][j] = weight
212             self.adj[j][i] = weight
213             self.n_edges += 1
214
215     def get_neighbors(self, i: int) -> Dict[int, float]:
216         return self.adj[i]
217
```

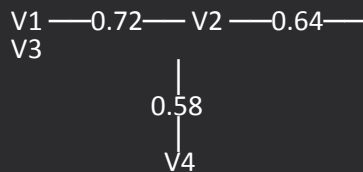
Threshold Adaptativo

- O threshold adaptativo define automaticamente a similaridade mínima para criar arestas no grafo, ajustando-se às características de cada artigo. Isso evita grafos excessivamente densos ou esparsos, que podem prejudicar o ranqueamento das frases.
- No projeto, esse ajuste pode ser feito a partir da distribuição das similaridades entre as frases, usando estratégias como média + desvio padrão, ou por densidade-alvo, escolhendo um valor de corte que mantenha uma proporção adequada de arestas.
- Também é possível definir esse threshold manualmente.



Implementação própria para ranqueamento

- Implementado em **graph_summarizer.py**, sem usar PageRank de bibliotecas externas.
- Cada frase começa com score inicial **1 / N**.
- A cada iteração, recebe importância das frases vizinhas.
- A contribuição considera o peso da aresta: maior similaridade = maior influência.
- Retorna também metadados: convergência, iterações, delta final e soma dos scores.



Redução de redundância no resumo

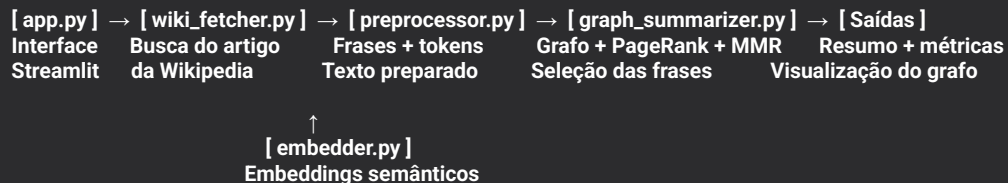
- Após o PageRank, o sistema cria um conjunto de frases candidatas com **heapq**.
- O MMR escolhe frases relevantes, mas penaliza frases parecidas com as já selecionadas.
- Fórmula conceitual:
- No projeto, **alpha = 0.85** por padrão.
- No final, as frases são reordenadas pela posição original no artigo.

$$\text{MMR} = \alpha * \text{importância} - (1 - \alpha) * \text{redundância}$$

Candidatas PageRank	MMR	Resumo final
F1 — alta relevância ↴	✓	F1
F2 — parecida com F1	— filtro —▶	✗ F2 redundante
F3 — complementar ↵	✓	F3

Backend modular + Streamlit

- **app.py**: interface Streamlit e integração com o usuário.
- **wiki_fetcher.py**: busca artigos da Wikipedia e usa cache em **dict**.
- **preprocessor.py**: segmentação, tokenização, stopwords e lematização.
- **embedder.py**: geração de embeddings semânticos.
- **graph_summarizer.py**: grafo, similaridades, PageRank, heap e MMR.
- **visualization.py**: visualização do grafo.



Fluxo de uso da aplicação

- O usuário informa uma URL ou escolhe um artigo exemplo.
- Seleciona idioma, nível do resumo e método de similaridade.
- Pode ajustar threshold, damping e estratégia de seleção.
- A aplicação mostra resumo, métricas, frases selecionadas, palavras-chave e grafo opcional.

Interface

Configurações

Nível de resumo

- ☐ ⚡ Essencial
- ☒ 📄 Estudo Rápido
- ☐ 🔍 Revisão Detalhada
- ☐ 📑 Personalizado (%)

Ideal para revisão em poucos minutos.

Idioma da busca

pt

Método de similaridade

- ☒ Embeddings semânticos (recomendado)
- ☐ Jaccard (palavras comuns)
- ☐ TF-IDF + Cosseno

Usa o modelo `paraphrase-multilingual-MiniLM-L12-v2` (~120 MB).
Baixado automaticamente na primeira execução.

> Avançado

- ☐ Exibir visualização do grafo
- ☒ Exibir frases com pontuação
- ☒ Exibir palavras-chave

Deploy



WikiSummarizer

Sumarização extrativa de artigos da Wikipedia usando **grafos** e **PageRank**.

URL ou título do artigo

https://pt.wikipedia.org/wiki/Inteligência_artificial ou Grafos

URL

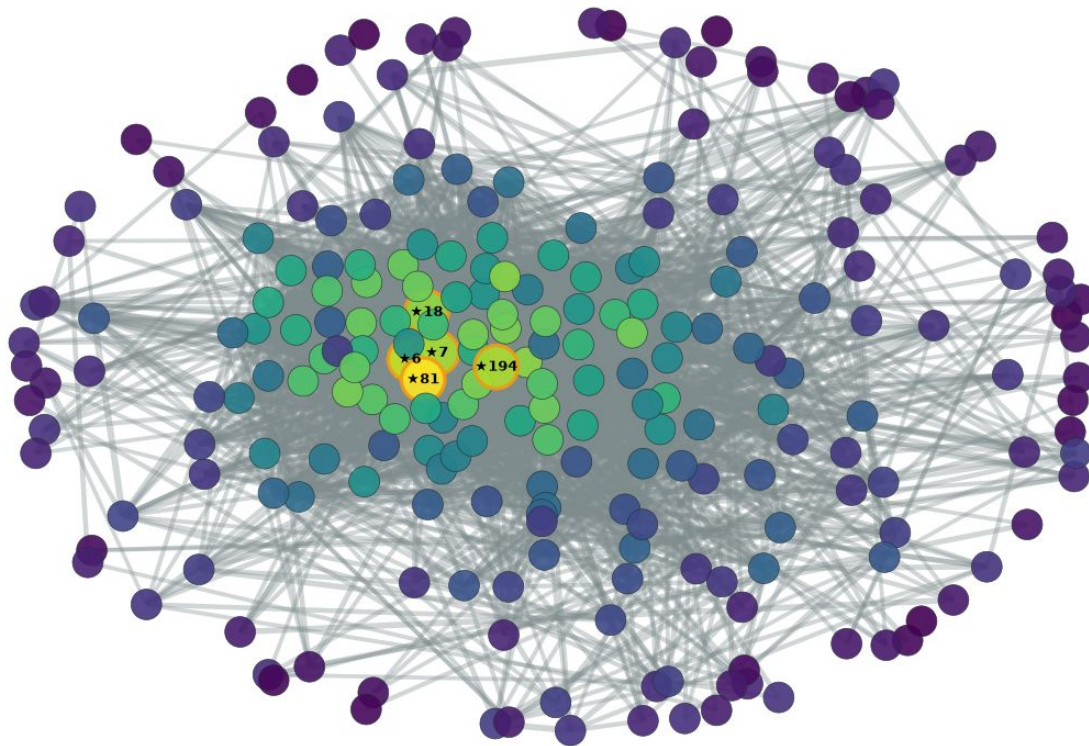
Gerar resumo

NetworkX apenas para visualização

- O grafo real é construído com lista de adjacência própria.
- NetworkX não calcula PageRank nem cria o grafo principal.
- Ele apenas desenha os nós e arestas para facilitar a demonstração.
- Nós maiores/destacados representam frases selecionadas no resumo.

Visualização

Grafo de Frases — 231 vértices | 3188 arestas | densidade 0.120



Comparação de métricas e qualidade

- Embeddings: melhor para conectar frases semanticamente próximas.
- TF-IDF: baseline lexical ponderado.
- Jaccard: baseline simples por sobreposição de tokens.
- Grafos muito densos reduzem a seletividade do PageRank.
- Grafos muito esparsos podem isolar frases importantes.

Os métodos mudam a forma do grafo e, por consequência, o resumo.

Embeddings
+ Semântico
+ Melhor cobertura

TF-IDF
+ Termos importantes
+ Baseline forte

Jaccard
+ Simples
+ Comparação básica

Testes

35 testes aprovados

- Similaridade Jaccard.
- Similaridade TF-IDF + cosseno.
- Construção do grafo.
- Estatísticas do grafo.
- PageRank.
- Heap para top-k frases.
- Threshold adaptativo.
- MMR.
- Pipeline completo.

Similaridade
Grafo
PageRank
Heap
Threshold
MMR
Pipeline

```
(.venv) tiago@tiago-IdeaPad-3-15ALC6:~/UnB/7 Semestre/EDA 2/EDA2-Sumarizacao$ python -m unittest str.test.test_wikisummarizer -v
```

```
-----  
Ran 35 tests in 3.212s
```

```
OK
```

```
❏ (.venv) tiago@tiago-IdeaPad-3-15ALC6:~/UnB/7 Semestre/EDA 2/EDA2-Sumarizacao$
```

Apoio para documentação e revisão

- Usada como apoio para documentação, revisão conceitual e organização da apresentação
- Usada para auxiliar a criação da interface do Streamlit
- Usada para formatação e revisão de código.
- Não substituiu a implementação dos algoritmos principais.
- Grafo, PageRank, heap, threshold e seleção das frases foram implementados no projeto.
-

Conclusão

Principais pontos

- Projeto feito em Python.
- Modela artigos como grafos de frases.
- Implementa lista de adjacência própria.
- Usa **dict**/hash table e **heapq**.
- Implementa PageRank ponderado.
- Permite escolher entre embeddings, TF-IDF e Jaccard.
- Gera resumo extrativo com métricas e visualização.

O WikiSummarizer transforma artigos longos em resumos extrativos usando grafos como estrutura central.